

Simple Command Sequencer Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user-friendly way of defining named command sequences for the Simple Command Sequencer component. The generator takes a YAML model file as input which declares a suite of sequences – each a named, ordered list of sub-commands (and optional sleeps) – and autocodes four things from it: the Ada sequence step tables, bundled with the suite’s two frame-pool sizes into a Config constant consumed by the sequencer’s Init routine; a set of first-class per-sequence wrapper commands so that each sequence appears on the sequencer instance as its own typed, operator-callable command; an operator-side command builder package for those wrapper commands; and the record type of the instance’s periodic summary packet.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. Model files must be named in the form *optional_name.assembly_name.command_sequences.yaml*, where *optional_name* is the specific name of this set of sequences (only needed if more than one suite is defined in the assembly) and *assembly_name* is the assembly the sequences will be used in. Generally this file is created in the same directory or near to the assembly model file. The schema is commented to show each available YAML key and what it accomplishes; even without knowing the specifics of pykwalify schemas, you should be able to glean the available structure from the file below.

```
1  # Schema for command_sequences YAML files
2  # This defines the structure and validation rules
3
4  type: map
5  mapping:
6    description:
7      required: false
8      type: str
9      desc: |
10         Description of this command sequences package. This will appear in
11         the generated Ada code comments.
12
13    preamble:
14      required: false
15      type: str
16      desc: |
17         Custom preamble text to include in the generated Ada package.
18
19    num_waiting_for_response_frames:
20      required: false
21      type: int
22      range: { min: 0 }
```

```

23     default: 0
24     desc: |
25         Frames for sequences that wait for their sub-commands' responses
26         (wait_for_command_completion true), one concurrently running sequence
27         per frame. These take frame ids 0 .. num_waiting_for_response_frames - 1.
28         ↪ Must be
29         at least 1 if any sequence in the suite waits, and exactly 0 if none
30         does; both are enforced by the model. Defaults to 0.
31
32 num_non_waiting_for_response_frames:
33     required: false
34     type: int
35     range: { min: 0 }
36     default: 0
37     desc: |
38         Frames for sequences that do not wait for their sub-commands' responses
39         (wait_for_command_completion false), one concurrently running sequence
40         per frame. These take the frame ids after the waiting-for-response frames.
41         ↪ Must be
42         at least 1 if any sequence in the suite does not wait, and exactly 0 if
43         none does; both are enforced by the model. Defaults to 0.
44
45     The pools are separate so a response still in flight to a finished
46     non-waiting-for-response run can never wake a frame that is parked
47     ↪ waiting. The two
48     counts together size the suite's generated summary packet type (one
49     Sequence_Frame_Summary per frame); their sum must be at least 1 and at
50     most the number of summaries that fit one packet buffer, enforced by
51     the model.
52
53 command_timeout_ms:
54     required: false
55     type: int
56     range: { min: 1, max: 2147483647 }
57     desc: |
58         Default timeout, in milliseconds, that the sequencer will wait for
59         a sub-command's response before declaring the sequence failed. Used
60         for any sequence in this suite that does not specify its own
61         command_timeout_ms. Bounded so the count always fits an
62         Ada.Real_Time.Time_Span.
63
64 with:
65     required: false
66     type: seq
67     desc: |
68         List of additional Ada packages to "with" in the generated sequences
69         package. Use this when a step's arg expression references a package
70         that is not pulled in automatically, e.g. an enum package for a
71         qualified enumeration literal.
72     sequence:
73         - type: str
74
75 sequences:
76     required: true
77     type: seq
78     desc: |
79         List of command sequence definitions. Each sequence defines a named
80         command that executes a series of sub-commands.
81     sequence:
82         - type: map
83           mapping:

```

```

81     name:
82         required: true
83         type: str
84         desc: |
85             Name of this sequence. This will become a command that can be
86             executed. Must follow Ada naming conventions.
87
88     description:
89         required: false
90         type: str
91         desc: |
92             Description of what this sequence does. Will appear in generated
93             code and documentation.
94
95     arg_type:
96         required: false
97         type: str
98         desc: |
99             The Ada type for this sequence's argument. If not specified,
100             the sequence takes no arguments. Must be a valid Ada type name,
101             possibly qualified (e.g., "Power_Mode.T" or "Packed_U16.T").
102
103     wait_for_command_completion:
104         required: false
105         type: bool
106         default: true
107         desc: |
108             Whether to wait for each sub-command's response before executing
109             the next step. Applies to every command step of the sequence.
110
111     continue_on_failure:
112         required: false
113         type: bool
114         default: false
115         desc: |
116             If true, the sequence will continue executing even if a
117             ↪ sub-command
118             fails. If false, the sequence will abort on the first failure.
119
120             A sub-command failure is only visible when the sequence waits for
121             its responses, so setting this key to false on a sequence with
122             wait_for_command_completion: false is rejected: such a sequence
123             could never abort.
124
125     response_behavior:
126         required: false
127         type: str
128         enum: ['send_after_sequence_start',
129             ↪ 'send_after_sequence_completion']
130         desc: |
131             When the sequencer replies to this sequence's own command:
132             send_after_sequence_start (the default) acknowledges as soon as
133             the sequence is started; send_after_sequence_completion defers
134             the reply until the sequence finishes, carrying its final
135             success or failure. Static per-sequence configuration; the
136             generic Run_Sequence command uses it too.
137
138     command_timeout_ms:
139         required: false
140         type: int
141         range: { min: 1, max: 2147483647 }

```

```

140     desc: |
141         Override the suite-level default timeout for sub-command
142         responses, in milliseconds, for this sequence only. Bounded
143         so the count always fits an Ada.Real_Time.Time_Span.
144
145 sequence:
146     required: true
147     type: seq
148     desc: |
149         The ordered list of steps to execute. Each step is either a
150         command dispatch (`command:` + optional `arg:`) or an in-sequence
151         sleep (`sleep_ms:` + integer milliseconds). The two forms are
152         mutually exclusive; exactly one must be present per step.
153         Validation is enforced in the model loader.
154     sequence:
155         - type: map
156           mapping:
157             command:
158                 required: false
159                 type: str
160                 desc: |
161                     The command to execute in the format
162                     ↪ "Component.Command_Name".
163                     The component must be a valid component instance name in
164                     ↪ your
165                     assembly, and the command must exist on that component.
166                     Mutually exclusive with `sleep:`.
167             arg:
168                 required: false
169                 type: str
170                 desc: |
171                     The argument for this command: either a reference into the
172                     parent sequence's argument ("Arg", or a field path such as
173                     "Arg.Power_Mode"), resolved per call, or a static Ada
174                     expression that does not mention Arg, baked in at build
175                     time. Mixing the two in one expression is rejected. If the
176                     command takes no argument, omit this field. Only valid
177                     alongside `command:`.
178
179                     Examples:
180                     - "Arg" - pass through the entire sequence argument
181                     - "Arg.Power_Mode" - pass a field from the sequence
182                       ↪ argument
183                     - "(Mode => Low, Enable => True)" - construct a record
184                       ↪ from literals
185                     - "42" - a literal value
186
187             sleep_ms:
188                 required: false
189                 type: text
190                 desc: |
191                     Milliseconds to pause the sequence. Not bus-routed; the
192                     sequencer frame parks itself for the duration without
193                     issuing any command. Mutually exclusive with
194                     `command:`/`arg:`.
195
196                     Either a static integer (0 .. 2**31 - 1, so it always fits
197                     an Ada.Real_Time.Time_Span; enforced by the model), or a
198                     dynamic reference into the sequence's argument using the
199                     same "Arg[.field]..." syntax as dynamic command args. A

```

```

197         dynamic sleep's referenced field must be a
198         Packed_Natural.T, which the resolver validates at
199         execution time.
200
201         Examples:
202         - sleep_ms: 3000 # sleep 3 seconds
203         - sleep_ms: Arg.Settle_Time # duration from the sequence
204           ↪ argument
205
206         # Additional validation rules (enforced in the model Python code):
207         # - Command format must be "Component.Command" with valid identifiers, or
208         #   the reserved name "Sleep"
209         # - Referenced types in arg_type must exist in the system
210         # - Argument expressions must be valid Ada syntax
211         # - Component and command names must match instances in the assembly

```

3 Example Input

The following is an example command sequences input YAML file. It defines twelve sequences exercising the main features: statically-armed commands and static sleeps, typed argument records whose fields are dispatched as dynamic arguments to sub-commands (including through non-byte-aligned packed fields and validatable records), a dynamic sleep whose duration comes from the sequence argument, no-wait direct dispatch, a sequence that calls another sequence as a sub-sequence step, and sequences whose command response is statically deferred until completion. This example adheres to the schema shown in the previous section, and is commented to give clarification.

Note the optional top-level `with:` key, which lists additional Ada packages to “with” in the generated package. A step’s `arg:` expression is emitted verbatim into the generated Ada, so any package it references that is not pulled in automatically – such as an enum package needed to qualify an enumeration literal – must be listed here.

```

1  # Command Sequences Definition
2  # This file defines composite commands that execute sequences of sub-commands
3
4  description: Example command sequences exercising the generator's features
5
6  # Four frames in two pools: waiting sequences run on frames 0 and 1,
6  ↪ non-waiting-for-response
7  # sequences on frames 2 and 3, and the generated summary packet type carries
8  # four frame summaries.
9  num_waiting_for_response_frames: 2
10 num_non_waiting_for_response_frames: 2
11
12 command_timeout_ms: 10000 # default wait for sub-command responses
13
14 sequences:
15   # Simple sequence with no arguments
16   - name: Sequence_A
17     description: Execute commands on component A
18     wait_for_command_completion: true
19     continue_on_failure: true
20
21     sequence:
22       - command: Component_A.Command_1
23       - command: Component_A.Command_2
24       arg: (Seconds => 3, Subseconds => 14)
25       - sleep_ms: 3000
26       - command: Component_A.Command_3

```

```

27     arg: (Value => 3)
28
29 - name: Sequence_B
30   description: Perform generic arguments with component A and B
31   wait_for_command_completion: true
32   continue_on_failure: false
33   arg_type: Sequence_B_Arg.T
34
35   sequence:
36     - command: Component_A.Command_3
37       arg: Arg.Component_A_Arg
38     - command: Component_B.Command_3
39       arg: Arg.Component_B_Arg
40
41 - name: Sequence_C
42   description: No-wait sequence for direct-dispatch testing
43   wait_for_command_completion: false
44
45   sequence:
46     - command: Component_A.Command_1
47     - command: Component_A.Command_1
48
49 - name: Sequence_D
50   description: Sequence with a dynamic sleep duration taken from the sequence
51   ↪ argument
52   wait_for_command_completion: true
53   continue_on_failure: false
54   arg_type: Packed_Natural.T
55
56   sequence:
57     - sleep_ms: Arg
58
59   # Calls another sequence. Each sequence is a command on the sequencer, so a
60   # sub-sequence step is a command step targeting the sequencer's own command.
61   # Sequence_C replies on start, so this sequence resumes as soon as it starts.
62 - name: Sequence_E
63   description: Call Sequence_C as a sub-sequence, then run a regular command
64   wait_for_command_completion: true
65   continue_on_failure: false
66
67   sequence:
68     - command: Sequencer.Sequence_C
69     - command: Component_A.Command_1
70
71   # Command response deferred until completion: the operator's reply carries
72   # the final success/failure instead of a start acknowledgement.
73 - name: Sequence_F
74   description: No-wait sequence with a statically deferred command response
75   wait_for_command_completion: false
76   response_behavior: send_after_sequence_completion
77
78   sequence:
79     - command: Component_A.Command_1
80
81   # Deferred response and waits on its sub-command, so the reply arrives when
82   # the sub-command completes (or the sequence aborts, times out, or is killed).
83 - name: Sequence_G
84   description: Waiting sequence with a statically deferred command response
85   wait_for_command_completion: true
86   continue_on_failure: false
87   response_behavior: send_after_sequence_completion

```

```

87
88     sequence:
89         - command: Component_A.Command_1
90
91     # Dynamic-arg repacking: the argument's leading 4-bit pad puts both leaf
92     # fields at non-byte-aligned offsets, so byte-offset slicing would corrupt
93     ↪ them.
94
95     - name: Sequence_H
96       description: Dynamic steps traversing misaligned 8-bit and 16-bit packed
97       ↪ fields
98
99       wait_for_command_completion: true
100      continue_on_failure: false
101      arg_type: Nibble_Test_Args.T
102
103      sequence:
104          - command: Component_A.Command_8
105            arg: Arg.Wide
106          - command: Component_A.Command_7
107            arg: Arg.Nibble
108
109      # First step is a static sleep, so wake-time overflow can be exercised
110      # without a command step's response deadline overflowing first.
111
112      - name: Sequence_I
113        description: A lone static sleep, for wake-time overflow coverage
114        wait_for_command_completion: true
115        continue_on_failure: false
116
117      sequence:
118          - sleep_ms: 3000
119
120      # The two sequences below take Validated_Args, whose Mode enumeration can
121      # fail validation, for the invalid-dynamic-argument failure paths.
122
123      - name: Sequence_J
124        description: Dynamic command argument resolved from a validatable record
125        wait_for_command_completion: true
126        continue_on_failure: false
127        arg_type: Validated_Args.T
128
129      sequence:
130          - command: Component_A.Command_3
131            arg: Arg.Duration
132
133      - name: Sequence_K
134        description: Dynamic sleep resolved from a validatable record
135        wait_for_command_completion: true
136        continue_on_failure: false
137        arg_type: Validated_Args.T
138
139      sequence:
140          - sleep_ms: Arg.Sleep_Duration
141
142      # Non-waiting-for-response sequence that stays busy across a tick (the sleep
143      ↪ parks it), so a
144      # test can fill the non-waiting-for-response pool.
145
146      - name: Sequence_L
147        description: Non-waiting-for-response sequence with a sleep, for
148        ↪ non-waiting-for-response pool exhaustion coverage
149        wait_for_command_completion: false
150
151      sequence:
152          - sleep_ms: 3000

```

4 Example Output

The example input above produces the following Ada outputs. The package spec declares the `To_Arg` buffer-padding helper, the per-dynamic-step Resolver functions, a step array constant per sequence, the `Sequences_Table` constant (plus a stable `Sequences` access pointer), and the suite's `Config` constant, which bundles the table access with the suite's `num_waiting_for_response_frames` and `num_non_waiting_for_response_frames`. The package body implements the Resolver functions: each validates the sequence's per-call argument buffer through the input type's generated `Validation.Valid`, deserializes it through `Serialization.From_Byte_Array`, and re-serializes the addressed field as the step's argument buffer (a sub-command's argument, or a `Packed_Natural` millisecond count for a dynamic sleep). Resolvers exist only for dynamic steps – a suite of purely static sequences declares none, and its body degenerates to `pragma No_Body;`.

The `Config` constant should be passed into the Simple Command Sequencer's `Init` procedure during assembly initialization, e.g. in the assembly YAML:

```
- type: Simple_Command_Sequencer
  name: My_Sequencer
  init:
    - "Config => Test_Assembly_Command_Sequences_Example_Sequences.Config"
```

```
1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Example command sequences exercising the generator's features
4  with Simple_Sequencer_Types; use Simple_Sequencer_Types;
5  with Test_Assembly_Commands; use Test_Assembly_Commands;
6  with Command_Types;
7  with Basic_Types;
8  with Sequence_Enums;
9  with Ada.Real_Time;
10 with Sys_Time_32;
11 with Packed_U32;
12 with Sequence_B_Arg;
13 with Packed_Natural;
14 with Nibble_Test_Args;
15 with Packed_U16;
16 with Packed_Nibble;
17 with Validated_Args;
18 package Test_Assembly_Command_Sequences_Example_Sequences is
19   -- Make the "&" operator on Byte_Array visible for To_Arg below.
20   use type Basic_Types.Byte_Array;
21
22   -----
23   -- Pad a serialized argument out to a full command argument buffer. An
24   -- expression function so the step arrays below can call it at elaboration.
25   -----
26   function To_Arg (Bytes : Basic_Types.Byte_Array) return
27     ↪ Command_Types.Command_Arg_Buffer_Type is
28     (Command_Types.Command_Arg_Buffer_Type (Bytes & Basic_Types.Byte_Array'(1
29       ↪ .. Command_Types.Command_Arg_Buffer_Type'Length - Bytes'Length =>
30       ↪ 0)));
31   -----
32   -- Resolvers - one per dynamic step, each extracting one field of the
33   -- sequence argument.
34   -----
35   function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
```

```

33 function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
   ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
34 function Sequence_D_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
   ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
35 function Sequence_H_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
   ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
36 function Sequence_H_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
   ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
37 function Sequence_J_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
   ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
38 function Sequence_K_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
   ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean;
39 -----
40 -- Step arrays - one constant per sequence.
41 -----
42 Sequence_A_Steps : aliased constant Step_Array :=
43 [
44   0 =>
45     (Kind      => Command_Step,
46      Id        => Component_A_Command_1,
47      Arg_Length => 0,
48      Arg        => [others => 0]),
49   1 =>
50     (Kind      => Command_Step,
51      Id        => Component_A_Command_2,
52      Arg_Length => Sys_Time_32.Serialization.Serialized_Length,
53      Arg        => To_Arg (Sys_Time_32.Serialization.To_Byte_Array
   ↪ ((Seconds => 3, Subseconds => 14)))),
54   2 =>
55     (Kind      => Sleep,
56      Id        => 0,
57      Arg_Length => 0,
58      Sleep_Arg  => 3000),
59   3 =>
60     (Kind      => Command_Step,
61      Id        => Component_A_Command_3,
62      Arg_Length => Packed_U32.Serialization.Serialized_Length,
63      Arg        => To_Arg (Packed_U32.Serialization.To_Byte_Array ((Value
   ↪ => 3))))
64 ];
65 Sequence_B_Steps : aliased constant Step_Array :=
66 [
67   0 =>
68     (Kind      => Runtime_Argument_Command_Step,
69      Id        => Component_A_Command_3,
70      Arg_Length => Packed_U32.Serialization.Serialized_Length,
71      Resolver   => Sequence_B_Step_0_Resolver_T'Access),
72   1 =>
73     (Kind      => Runtime_Argument_Command_Step,
74      Id        => Component_B_Command_3,
75      Arg_Length => Packed_U32.Serialization.Serialized_Length,
76      Resolver   => Sequence_B_Step_1_Resolver_T'Access)
77 ];
78 Sequence_C_Steps : aliased constant Step_Array :=
79 [
80   0 =>
81     (Kind      => Command_Step,
82      Id        => Component_A_Command_1,
83      Arg_Length => 0,
84      Arg        => [others => 0]),
85   1 =>

```

```

86         (Kind      => Command_Step,
87          Id        => Component_A_Command_1,
88          Arg_Length => 0,
89          Arg        => [others => 0])
90     ];
91     Sequence_D_Steps : aliased constant Step_Array :=
92     [
93         0 =>
94             (Kind      => Runtime_Sleep,
95              Id        => 0,
96              Arg_Length => Packed_Natural.Serialization.Serialized_Length,
97              Sleep_Resolver => Sequence_D_Step_0_Resolver_T'Access)
98     ];
99     Sequence_E_Steps : aliased constant Step_Array :=
100    [
101        0 =>
102            (Kind      => Command_Step,
103             Id        => Sequencer_Sequence_C,
104             Arg_Length => 0,
105             Arg        => [others => 0]),
106        1 =>
107            (Kind      => Command_Step,
108             Id        => Component_A_Command_1,
109             Arg_Length => 0,
110             Arg        => [others => 0])
111    ];
112    Sequence_F_Steps : aliased constant Step_Array :=
113    [
114        0 =>
115            (Kind      => Command_Step,
116             Id        => Component_A_Command_1,
117             Arg_Length => 0,
118             Arg        => [others => 0])
119    ];
120    Sequence_G_Steps : aliased constant Step_Array :=
121    [
122        0 =>
123            (Kind      => Command_Step,
124             Id        => Component_A_Command_1,
125             Arg_Length => 0,
126             Arg        => [others => 0])
127    ];
128    Sequence_H_Steps : aliased constant Step_Array :=
129    [
130        0 =>
131            (Kind      => Runtime_Argument_Command_Step,
132             Id        => Component_A_Command_8,
133             Arg_Length => Packed_U16.Serialization.Serialized_Length,
134             Resolver  => Sequence_H_Step_0_Resolver_T'Access),
135        1 =>
136            (Kind      => Runtime_Argument_Command_Step,
137             Id        => Component_A_Command_7,
138             Arg_Length => Packed_Nibble.Serialization.Serialized_Length,
139             Resolver  => Sequence_H_Step_1_Resolver_T'Access)
140    ];
141    Sequence_I_Steps : aliased constant Step_Array :=
142    [
143        0 =>
144            (Kind      => Sleep,
145             Id        => 0,
146             Arg_Length => 0,

```

```

147         Sleep_Arg => 3000)
148     ];
149     Sequence_J_Steps : aliased constant Step_Array :=
150     [
151         0 =>
152             (Kind      => Runtime_Argument_Command_Step,
153              Id        => Component_A_Command_3,
154              Arg_Length => Packed_U32.Serialization.Serialized_Length,
155              Resolver  => Sequence_J_Step_0_Resolver_T'Access)
156     ];
157     Sequence_K_Steps : aliased constant Step_Array :=
158     [
159         0 =>
160             (Kind      => Runtime_Sleep,
161              Id        => 0,
162              Arg_Length => Packed_Natural.Serialization.Serialized_Length,
163              Sleep_Resolver => Sequence_K_Step_0_Resolver_T'Access)
164     ];
165     Sequence_L_Steps : aliased constant Step_Array :=
166     [
167         0 =>
168             (Kind      => Sleep,
169              Id        => 0,
170              Arg_Length => 0,
171              Sleep_Arg => 3000),
172         1 =>
173             (Kind      => Command_Step,
174              Id        => Component_A_Command_1,
175              Arg_Length => 0,
176              Arg       => [others => 0])
177     ];
178     -----
179     -- Sequences_Table
180     -----
181     Sequences_Table : aliased constant Sequences_Type :=
182     [
183         0 =>
184             (Wait_For_Cmd_Resp  => True,
185              Abort_On_Failed_Cmd => False,
186              Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
187              Response_Behavior  => Sequence_Enums.Sequence_Response_Behavior.Send]
188             ↪ _After_Sequence_Start,
189             Arg_Length          => 0,
190             Steps               => Sequence_A_Steps'Access),
191         1 =>
192             (Wait_For_Cmd_Resp  => True,
193              Abort_On_Failed_Cmd => True,
194              Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
195              Response_Behavior  => Sequence_Enums.Sequence_Response_Behavior.Send]
196             ↪ _After_Sequence_Start,
197             Arg_Length          => Sequence_B_Arg.Serialization.Serialized_Length,
198             Steps               => Sequence_B_Steps'Access),
199         2 =>
200             (Wait_For_Cmd_Resp  => False,
201              Abort_On_Failed_Cmd => True,
202              Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
203              Response_Behavior  => Sequence_Enums.Sequence_Response_Behavior.Send]
204             ↪ _After_Sequence_Start,
205             Arg_Length          => 0,
206             Steps               => Sequence_C_Steps'Access),
207         3 =>

```

```

205     (Wait_For_Cmd_Resp  => True,
206     Abort_On_Failed_Cmd => True,
207     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
208     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
209     ↪ _After_Sequence_Start,
210     Arg_Length          => Packed_Natural.Serialization.Serialized_Length,
211     Steps               => Sequence_D_Steps'Access),
212 4 =>
213     (Wait_For_Cmd_Resp  => True,
214     Abort_On_Failed_Cmd => True,
215     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
216     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
217     ↪ _After_Sequence_Start,
218     Arg_Length          => 0,
219     Steps               => Sequence_E_Steps'Access),
220 5 =>
221     (Wait_For_Cmd_Resp  => False,
222     Abort_On_Failed_Cmd => True,
223     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
224     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
225     ↪ _After_Sequence_Completion,
226     Arg_Length          => 0,
227     Steps               => Sequence_F_Steps'Access),
228 6 =>
229     (Wait_For_Cmd_Resp  => True,
230     Abort_On_Failed_Cmd => True,
231     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
232     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
233     ↪ _After_Sequence_Completion,
234     Arg_Length          => 0,
235     Steps               => Sequence_G_Steps'Access),
236 7 =>
237     (Wait_For_Cmd_Resp  => True,
238     Abort_On_Failed_Cmd => True,
239     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
240     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
241     ↪ _After_Sequence_Start,
242     Arg_Length          =>
243     ↪ Nibble_Test_Args.Serialization.Serialized_Length,
244     Steps               => Sequence_H_Steps'Access),
245 8 =>
246     (Wait_For_Cmd_Resp  => True,
247     Abort_On_Failed_Cmd => True,
248     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
249     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
250     ↪ _After_Sequence_Start,
251     Arg_Length          => 0,
252     Steps               => Sequence_I_Steps'Access),
253 9 =>
254     (Wait_For_Cmd_Resp  => True,
255     Abort_On_Failed_Cmd => True,
256     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
257     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
258     ↪ _After_Sequence_Start,
259     Arg_Length          => Validated_Args.Serialization.Serialized_Length,
260     Steps               => Sequence_J_Steps'Access),
261 10 =>
262     (Wait_For_Cmd_Resp  => True,
263     Abort_On_Failed_Cmd => True,
264     Command_Timeout    => Ada.Real_Time.Milliseconds (10000),
265     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send,
266     ↪ _After_Sequence_Start,

```

```

258     Arg_Length      => Validated_Args.Serialization.Serialized_Length,
259     Steps           => Sequence_K_Steps'Access),
260 11 =>
261     (Wait_For_Cmd_Resp  => False,
262     Abort_On_Failed_Cmd => True,
263     Command_Timeout     => Ada.Real_Time.Milliseconds (10000),
264     Response_Behavior   => Sequence_Enums.Sequence_Response_Behavior.Send]
    ↪ _After_Sequence_Start,
265     Arg_Length         => 0,
266     Steps              => Sequence_L_Steps'Access)
267 ];
268
269 Sequences : constant Sequences_Access := Sequences_Table'Access;
270
271 -- Passed to Simple_Command_Sequencer.Init. The
    ↪ Test_Assembly_Command_Sequences_Example_Sequences_Summary_Record
272 -- packet type is generated from the same two pool sizes, so the frame pools
273 -- and the packet layout cannot disagree.
274 Config : constant Sequencer_Config :=
275     (Sequences => Sequences_Table'Access,
276     Num_Waiting_For_Response_Frames => 2,
277     Num_Non_Waiting_For_Response_Frames => 2);
278 end Test_Assembly_Command_Sequences_Example_Sequences;

```

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  with Interfaces;
4  with Sequence_B_Arg.Validation;
5  with Packed_Natural.Validation;
6  with Nibble_Test_Args.Validation;
7  with Validated_Args.Validation;
8  package body Test_Assembly_Command_Sequences_Example_Sequences is
9
10     -----
11     -- Resolvers - one per dynamic step. Each validates and deserializes the
12     -- sequence argument, then returns the addressed field serialized (a
13     -- sub-command argument, or a Packed_Natural millisecond count for a sleep).
14     -----
15     function Sequence_B_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
16     Errant_Field : Interfaces.Unsigned_32;
17     Input : Sequence_B_Arg.T;
18     Valid : constant Boolean := Sequence_B_Arg.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1), Errant_Field);
19 begin
20     if Valid then
21         Input := Sequence_B_Arg.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
22         Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
    ↪ (Input.Component_A_Arg));
23     end if;
24     return Valid;
25 end Sequence_B_Step_0_Resolver_T;
26     function Sequence_B_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
27     Errant_Field : Interfaces.Unsigned_32;
28     Input : Sequence_B_Arg.T;
29     Valid : constant Boolean := Sequence_B_Arg.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1), Errant_Field);

```

```

30   begin
31       if Valid then
32           Input := Sequence_B_Arg.Serialization.From_Byte_Array (Bytes
33               ↪ (Bytes'First .. Bytes'First +
34               ↪ Sequence_B_Arg.Serialization.Serialized_Length - 1));
35           Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
36               ↪ (Input.Component_B_Arg));
37       end if;
38       return Valid;
39   end Sequence_B_Step_1_Resolver_T;
40   function Sequence_D_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
41       ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
42       Errant_Field : Interfaces.Unsigned_32;
43       Input : Packed_Natural.T;
44       Valid : constant Boolean := Packed_Natural.Validation.Valid (Bytes
45           ↪ (Bytes'First .. Bytes'First +
46           ↪ Packed_Natural.Serialization.Serialized_Length - 1), Errant_Field);
47   begin
48       if Valid then
49           Input := Packed_Natural.Serialization.From_Byte_Array (Bytes
50               ↪ (Bytes'First .. Bytes'First +
51               ↪ Packed_Natural.Serialization.Serialized_Length - 1));
52           Args := To_Arg (Packed_Natural.Serialization.To_Byte_Array (Input));
53       end if;
54       return Valid;
55   end Sequence_D_Step_0_Resolver_T;
56   function Sequence_H_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
57       ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
58       Errant_Field : Interfaces.Unsigned_32;
59       Input : Nibble_Test_Args.T;
60       Valid : constant Boolean := Nibble_Test_Args.Validation.Valid (Bytes
61           ↪ (Bytes'First .. Bytes'First +
62           ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1), Errant_Field);
63   begin
64       if Valid then
65           Input := Nibble_Test_Args.Serialization.From_Byte_Array (Bytes
66               ↪ (Bytes'First .. Bytes'First +
67               ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1));
68           Args := To_Arg (Packed_U16.Serialization.To_Byte_Array (Input.Wide));
69       end if;
70       return Valid;
71   end Sequence_H_Step_0_Resolver_T;
72   function Sequence_H_Step_1_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
73       ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
74       Errant_Field : Interfaces.Unsigned_32;
75       Input : Nibble_Test_Args.T;
76       Valid : constant Boolean := Nibble_Test_Args.Validation.Valid (Bytes
77           ↪ (Bytes'First .. Bytes'First +
78           ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1), Errant_Field);
79   begin
80       if Valid then
81           Input := Nibble_Test_Args.Serialization.From_Byte_Array (Bytes
82               ↪ (Bytes'First .. Bytes'First +
83               ↪ Nibble_Test_Args.Serialization.Serialized_Length - 1));
84           Args := To_Arg (Packed_Nibble.Serialization.To_Byte_Array
85               ↪ (Input.Nibble));
86       end if;
87       return Valid;
88   end Sequence_H_Step_1_Resolver_T;
89   function Sequence_J_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
90       ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is

```

```

71     Errant_Field : Interfaces.Unsigned_32;
72     Input : Validated_Args.T;
73     Valid : constant Boolean := Validated_Args.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1), Errant_Field);
74 begin
75     if Valid then
76         Input := Validated_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1));
77         Args := To_Arg (Packed_U32.Serialization.To_Byte_Array
    ↪ (Input.Duration));
78     end if;
79     return Valid;
80 end Sequence_J_Step_0_Resolver_T;
81 function Sequence_K_Step_0_Resolver_T (Bytes : Basic_Types.Byte_Array; Args :
    ↪ out Command_Types.Command_Arg_Buffer_Type) return Boolean is
82     Errant_Field : Interfaces.Unsigned_32;
83     Input : Validated_Args.T;
84     Valid : constant Boolean := Validated_Args.Validation.Valid (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1), Errant_Field);
85 begin
86     if Valid then
87         Input := Validated_Args.Serialization.From_Byte_Array (Bytes
    ↪ (Bytes'First .. Bytes'First +
    ↪ Validated_Args.Serialization.Serialized_Length - 1));
88         Args := To_Arg (Packed_Natural.Serialization.To_Byte_Array
    ↪ (Input.Sleep_Duration));
89     end if;
90     return Valid;
91 end Sequence_K_Step_0_Resolver_T;
92 end Test_Assembly_Command_Sequences_Example_Sequences;
93

```

5 Per-Sequence First-Class Commands

In addition to the sequence tables above, this same model drives the autocoded synthesis of one first-class command per declared sequence on the Simple Command Sequencer instance. Discovery is init-parameter-driven: at assembly build time the sequencer's commands model reads each instance's Config init parameter, derives the suite package named there (e.g. `Test_Assembly_Command_Sequences_Example_Sequences.Config`), and loads exactly that suite's model, injecting one command per declared sequence into the instance's command set. A naked component build (no assembly, hence no instance and no Config value) produces only the four static commands (`Run_Sequence`, `Kill_All_Sequences`, `Kill_Frame`, `Set_Summary_Packet_Period`); a suite merely present on the build path contributes nothing – only the suite an instance names in its Config adds commands, so two sequencer instances in one assembly can carry different suites. Each per-sequence command (e.g. `Sequence_B`) takes the sequence's own `arg_type` verbatim as its command argument (or no argument for an argless sequence) – a user-written, normally-registered type, so no argument records are generated or registered anywhere in the pipeline. At run time the component copies the received argument buffer unchanged into a generic `Run_Sequence_Arg.T` and dispatches through the existing `Run_Sequence` backbone using the sequence's runtime slot index. `Run_Sequence` rejects any argument whose length differs from the serialized length of the sequence's `arg_type` (recorded in the autocoded table) with an `Invalid_Sequence_Argument_Length` event; the response behavior comes from the sequence's static configuration in the same table (see Sequence Response Behavior below). A companion generated package, `<suite>_Commands`, carries the operator-side builder surface (per-sequence command id get-

ters and Command.T constructors) for unit tests and other on-board callers – split from the sequences package because the sequencer component itself never uses the builders. No per-sequence Ada needs to be hand-written:

```

1  -- Auto-generated by Adamant command sequences generator.
2  -- Do not edit by hand.
3  -- Example command sequences exercising the generator's features
4  --
5  -- Operator-side builders (id getters and Command.T constructors) for the
6  -- Test_Assembly_Command_Sequences_Example_Sequences suite's per-sequence
7  -- commands, for unit tests and on-board
8  -- callers. The sequencer component itself does not use this package.
9  with Command;
10 with Command_Types;
11 with Nibble_Test_Args;
12 with Packed_Natural;
13 with Sequence_B_Arg;
14 with Validated_Args;
15 package Test_Assembly_Command_Sequences_Example_Sequences_Commands is
16
17     -- Set Id_Base to the sequencer instance's command id base. Each sequence
18     -- command id is Id_Base + Num_Commands + declaration index.
19     type Instance is tagged private;
20
21     -- Set the sequencer instance's command id base so constructed commands
22     -- carry
23     -- ids that match what the component registered at runtime.
24     procedure Set_Id_Base (Self : in out Instance; Id_Base : in
25         Command_Types.Command_Id);
26
27     -- Set the operator source id stamped into the header of constructed commands
28     -- (defaults to 0). Mirrors the command suite's Set_Source_Id.
29     procedure Set_Source_Id (Self : in out Instance; Source_Id : in
30         Command_Types.Command_Source_Id);
31
32     function Get_Sequence_A_Id (Self : in Instance) return
33         Command_Types.Command_Id;
34     function Sequence_A (Self : in Instance) return Command.T;
35     function Get_Sequence_B_Id (Self : in Instance) return
36         Command_Types.Command_Id;
37     function Sequence_B (Self : in Instance; Arg : in Sequence_B_Arg.T) return
38         Command.T;
39     function Get_Sequence_C_Id (Self : in Instance) return
40         Command_Types.Command_Id;
41     function Sequence_C (Self : in Instance) return Command.T;
42     function Get_Sequence_D_Id (Self : in Instance) return
43         Command_Types.Command_Id;
44     function Sequence_D (Self : in Instance; Arg : in Packed_Natural.T) return
45         Command.T;
46     function Get_Sequence_E_Id (Self : in Instance) return
47         Command_Types.Command_Id;
48     function Sequence_E (Self : in Instance) return Command.T;
49     function Get_Sequence_F_Id (Self : in Instance) return
50         Command_Types.Command_Id;
51     function Sequence_F (Self : in Instance) return Command.T;
52     function Get_Sequence_G_Id (Self : in Instance) return
53         Command_Types.Command_Id;
54     function Sequence_G (Self : in Instance) return Command.T;
55     function Get_Sequence_H_Id (Self : in Instance) return
56         Command_Types.Command_Id;
57     function Sequence_H (Self : in Instance; Arg : in Nibble_Test_Args.T) return
58         Command.T;

```

```

44     function Get_Sequence_I_Id (Self : in Instance) return
      ↪ Command_Types.Command_Id;
45     function Sequence_I (Self : in Instance) return Command.T;
46     function Get_Sequence_J_Id (Self : in Instance) return
      ↪ Command_Types.Command_Id;
47     function Sequence_J (Self : in Instance; Arg : in Validated_Args.T) return
      ↪ Command.T;
48     function Get_Sequence_K_Id (Self : in Instance) return
      ↪ Command_Types.Command_Id;
49     function Sequence_K (Self : in Instance; Arg : in Validated_Args.T) return
      ↪ Command.T;
50     function Get_Sequence_L_Id (Self : in Instance) return
      ↪ Command_Types.Command_Id;
51     function Sequence_L (Self : in Instance) return Command.T;
52
53 private
54     type Instance is tagged record
55         Id_Base : Command_Types.Command_Id := 0;
56         Source_Id : Command_Types.Command_Source_Id := 0;
57     end record;
58 end Test_Assembly_Command_Sequences_Example_Sequences_Commands;

```

6 Sequences Calling Sequences

Because every sequence is a first-class command on the sequencer instance, a sequence step may invoke another sequence simply by targeting the sequencer's own synthesized command, e.g. `command: My_Sequencer.Sequence_C`. No special syntax or generator support is involved – the step resolves like any other `Component.Command` reference, provided the sequencer instance is declared in the assembly. At run time the sub-sequence command is emitted out the sequencer's `Command_T_Send` connector, routed through the command router, and arrives back at the sequencer's `Command_T_Recv_Async`, where it claims its own frame and executes concurrently with (and independently of) the calling sequence. If the calling sequence waits for command completion, the sub-sequence's command response (per its static `response_behavior` configuration, see below) is what wakes it. Nesting depth is naturally capped by the size of the frame pool the sub-sequence draws from (see Frame Pools below): a sub-sequence call when that pool has no idle frame is rejected with a `No_Frame_Available` event naming the sequence and the pool, and a `Failure` response to the calling frame. One shape is refused at generation time: a cycle of calls in which every sequence has `response_behavior: send_after_sequence_completion` and waits for command completion (a deferred sequence calling itself included). A waiting call into a deferred sequence parks the caller's frame until the callee finishes, so around such a cycle no run could ever finish and frames would be consumed until dispatch fails. A cycle that passes through a `send_after_sequence_start` sequence, or through a sequence that does not wait for command completion, completes one run per lap and is allowed, so a sequence may deliberately loop through itself either way.

7 Frame Pools

A sequencer instance owns two pools of frames, sized by the suite's `num_waiting_for_response_frames` and `num_non_waiting_for_response_frames`. A sequence with `wait_for_command_completion: true` always runs on a frame of the waiting-for-response pool (frame ids `0 .. num_waiting_for_response_frames - 1`); a sequence that does not wait always runs on a frame of the non-waiting-for-response pool (the ids after them). The split exists because sub-command responses are matched to a frame by source id alone and carry no run tag: when a non-waiting-for-response run ends, its frame goes idle with responses still in flight to it, and had the frame been handed to a sequence that then parked waiting on the same command id, the stale response would have woken the wrong step. A non-waiting-for-response frame never parks on a response, so a late response arriving there has nothing to wake, and a waiting-for-response frame only ever receives

responses to dispatches that waited. The remaining ways a response can arrive late on a waiting frame – a step that timed out, or a frame killed while parked – are fault paths, guarded as in the full command sequencer by the pending-command-id match. Within each pool frames are handed out round robin, so the frame released most recently is the last one of its pool reused. Each pool is sized exactly to its users: the model rejects a suite whose sequences need a frame from an empty pool, and equally a suite that gives frames to a pool no sequence draws from, since nothing could ever claim them. Both keys are optional and default to zero, so a suite whose sequences all wait declares only `num_waiting_for_response_frames`. Because the pools are static, idle frames of one pool cannot serve the other: `No_Frame_Available` reports the pool that was full.

8 Sequence Response Behavior

When the sequencer replies to the command that started a sequence is controlled by the `Sequence_Response_Behavior` enumeration. It is static per-sequence configuration: the optional `response_behavior` field on each sequence in the YAML, baked into the autocoded sequence table and used by both the synthesized per-sequence commands and the generic `Run_Sequence` command:

- `Send_After_Sequence_Start` (the conventional default) – the command response is sent immediately once the sequence has been accepted and started on a frame. This is the right choice for fire-and-forget operation and for sub-sequence calls where the caller should proceed as soon as the sub-sequence is running.
- `Send_After_Sequence_Completion` – the command response is withheld until the sequence ends, and its status reflects the outcome: `Success` when the final step completes, or `Failure` when the sequence is aborted by a failed sub-command, times out waiting for a sub-command response, fails to resolve or validate a dynamic argument, encounters an out-of-range sleep or timeout, or is halted by `Kill_All_Sequences` or `Kill_Frame`. The sequencer captures the originating command’s source and id per frame, so concurrent deferred sequences each reply to their own operator.

A sequence step may forward a field of the parent sequence’s argument to the sub-command it dispatches, using the `Arg.field.subfield` syntax in the step’s `arg`:. Each such reference produces a typed Resolver in the generated package: at run time the sequencer dispatches that Resolver, which first validates the per-call argument buffer through the input type’s generated `Validation.Valid` (a failure aborts the sequence with an `Invalid_Dynamic_Command_Argument` event), then deserializes it through the input type’s safe `Serialization.From_Byte_Array`, navigates to the addressed field, and re-serializes it as the sub-command’s argument buffer. The resolver’s signature accepts only a `Basic_Types.Byte_Array` – there is no ‘Address reinterpretation’ – so the dynamic-argument machinery is type-safe by construction; a resolver cannot overlay an arbitrary record onto raw memory, only deserialize a byte array through the input type’s generated deserializer.

9 Sleep Steps

A sequence step may be a sleep instead of a command: `sleep_ms`: pauses the executing frame without issuing any bus traffic. Sleep is not a bus command; it is interpreted intrinsically by the sequencer frame state machine. The duration takes two forms. A *static* sleep (`sleep_ms: 3000`) is a plain integer number of milliseconds fixed at build time; the model bounds it to $0 \dots 2^{31} - 1$ so it always fits an `Ada.Real_Time.Time_Span` by construction, leaving wake-time overflow of `Sys_Time` arithmetic as its only run-time failure. A *dynamic* sleep (`sleep_ms: Arg.Settle_Time`) instead references the sequence’s per-call argument with the same `Arg[.field]` syntax as dynamic command arguments; the referenced field must be a `Packed_Natural.T`, resolved at execution time through the same validating Resolver machinery, so the duration is range-checked as part of argument validation (a failure aborts the sequence with an `Invalid_Dynamic_Sleep_Argument` event) and always fits a `Time_Span`. A sleep whose wake-time computation overflows ends the sequence with a `Sequence_Out_Of_Range_Sleep` event and a `Failure` status to any pending deferred reply – continuing would leave the frame parked on a stale wake time.

10 Summary Packet Type

The sequencer emits a periodic `Summary_Packet` containing one `Sequence_Frame_Summary` record per sequence frame, in frame order. Its layout therefore depends on the suite's frame count, `num_waiting_for_response_frames` plus `num_non_waiting_for_response_frames` (waiting-for-response frames first), so the generator also autocodes the packet's ground/documentation record type, `<suite>_summary_record.record.yaml`, alongside the suite's other outputs. The flight software never imports this type – the component fills the packet buffer directly from its two frame pools – but ground tools and assembly documentation need it. At assembly load time the sequencer's packets model resolves an instance's `Summary_Packet` type to the summary record of the suite named by the instance's `Config` init parameter. Because the frame pools and the packet layout both come from the same two suite model values, they can never disagree; the model also rejects at build time any suite whose frame summaries would not fit in one packet buffer (the same bound the `Num_Concurrent-Sequences_Type` subtype enforces at `Init`). The example input above generates the following record model:

```
1 ---
2 description: Autocoded summary packet type for the
   ↳ Test_Assembly_Command-Sequences_Example-Sequences command sequences suite.
   ↳ Contains one Sequence_Frame_Summary record per sequence frame (4 in all, the
   ↳ 2 waiting-for-response frames followed by the 2 non-waiting-for-response
   ↳ frames), in frame order.
3 fields:
4   - name: Frame_0_Summary
5     description: "A summary of the state of sequence frame 0."
6     type: Sequence_Frame_Summary.T
7   - name: Frame_1_Summary
8     description: "A summary of the state of sequence frame 1."
9     type: Sequence_Frame_Summary.T
10  - name: Frame_2_Summary
11    description: "A summary of the state of sequence frame 2."
12    type: Sequence_Frame_Summary.T
13  - name: Frame_3_Summary
14    description: "A summary of the state of sequence frame 3."
15    type: Sequence_Frame_Summary.T
16
```